

Звіт

з ASM-проєкту

Стегня Віктора ПМі-44

Мета та основні завдання

1. Розробити графічний інтерфейс для роботи з ASM-кодом.
2. Реалізувати механізм відкриття та збереження файлів.
3. Додати можливість компіляції асемблерного коду в машинний код.
4. Реалізувати запуск коду через емуляцію виконання інструкцій.
5. Додати відображення результату виконання та журналювання подій.
6. Підготувати приклади, довідку та зручну форму використання програми.

Короткий опис реалізованої системи

У результаті роботи було створено застосунок на Python із використанням бібліотеки PySide6 для побудови GUI. Для роботи з асемблером використано:

- Keystone Engine для компіляції ASM-коду в машинний код;
- Unicorn Engine для емуляції виконання згенерованих інструкцій.

Програма дозволяє:

- вводити або редагувати ASM-код у текстовому полі;
- відкривати готові файли з розширеннями ``.asm`` та ``.txt``;
- зберігати написаний код у файл;
- компілювати код без запуску;
- зберігати результат компіляції у вигляді ``.bin`` або ``.txt``;
- запускати код в емуляторі;
- бачити текстовий результат виконання програми;
- переглядати логи роботи застосунку;
- користуватися вбудованою довідкою.

Основні компоненти проєкту

1. Графічний інтерфейс

У файлі `main.py` реалізовано головне вікно програми. Інтерфейс містить:

- верхню панель керування з кнопками `'Open'`, `'Save'`, `'Logs'`, `'Help'`;
- центральне поле для введення ASM-коду;
- кнопки `'Compile'` і `'Run'`;
- нижнє поле для відображення результату роботи програми.

Також реалізовано окремі вікна:

- вікно логів для перегляду повідомлень про виконання;
- вікно довідки з короткою інструкцією користувача.

2. Модуль роботи з файлами

У файлі `file_manager.py` реалізовано:

- відкриття ASM-файлів із файлової системи;
- зчитування тексту з файлу в редактор;
- збереження поточного вмісту редактора у файл.

Цей модуль дозволив відокремити файлову логіку від графічного інтерфейсу та зробити структуру проєкту більш зрозумілою.

3. Модуль компіляції та запуску

У файлі `runner.py` реалізовано основну логіку роботи з асемблерним кодом:

- компіляцію інструкцій у машинний код;
- підготовку пам'яті для виконання;
- запуск коду в емуляторі;
- перехоплення системних викликів;
- обробку виводу через `'write'`;
- завершення виконання через `'exit'`.

Таким чином, програма не просто зберігає код, а реально виконує його в контрольованому середовищі.

4. Оформлення та зручність користування

У проєкті також реалізовано:

- файл стилів ``style.qss`` для візуального оформлення;
- приклади ``examples/helloworld.txt`` та ``examples/loop.txt``;
- конфігурацію збірки ``main.spec``;
- готовий зібраний виконуваний файл ``dist/ASM runner.exe``.

Висновки

У ході виконання проєкту було розроблено навчальний застосунок для введення, компіляції та виконання програм на асемблері x86-64 з графічним інтерфейсом і базовими сервісними можливостями.